

The Four Types of Fixes A Taxonomy of Software Repair Methods

William L. Anderson

Computer Consoles, Inc.
Telephony Systems Group
1212 Pittsford-Victor Road
Pittsford, New York 14534
(716) 248-8200, (716) 482-5000

ABSTRACT

This paper describes criteria for management and control of software maintenance activity. Software maintenance is compared with industrial production; a convenient model for this process is a software assembly line. Software repair methods are classified into four types and contrasted in terms of three attributes:

1. the time required to complete repair;
2. the certification of the quality of the repair; and
3. the scope of side effects of the repair on the rest of the product.

Comparing these four methods elucidates the tradeoff between repair urgency and product quality that governs the management of software maintenance activities.

The Four Types of Fixes

A Taxonomy of Software Repair Methods

William L. Anderson

Computer Consoles, Inc.
Telephony Systems Group
1212 Pittsford-Victor Road
Pittsford, New York 14534
(716) 248-8200, (716) 482-5000

1. Introduction.

This paper describes a model of the software maintenance process that differentiates four repair methods. This classification exposes the risks and benefits associated with different software maintenance procedures, and thereby provides a basis for management control.

The fundamental model for software maintenance and production is that of an assembly line. [9, 11] Components and subcomponents are repaired and rebuilt at individual programmer workbenches. Finished parts are passed on to subsequent integration stations for assembly into larger units and regression testing. At the final station the entire product is built and certified.

Maintenance activities are initiated at any of the programmer

workbenches or integration stations. Four types of fixes (software repairs) are distinguished by the ability to certify the quality of the repaired product. Benefits and risks are explained for each repair method.

Finally, the advantage of differentiating repair procedures for management and control of software maintenance is explained.

2. Maintenance is not development, it's manufacturing.

Maintenance and distribution of a software product is not the same as initial development. Nearly 16 years ago, Ed David perceptively remarked that "in computing, the research, development, and production phases are often telescoped into one process.... In the cold light of day, we know that a step-by-step approach separating research and development from production is less risky and more likely to be successful." [2]

The difference between research and development and production is easily seen by analogy with the chemicals industry. Research and development of new products is carried out by chemists. The design and implementation of processes for efficient manufacture of specific products is the work of chemical engineers. Research and engineering

activities are necessary for successful commercial chemical products.

Both activities are needed for successful commercial software products.

Software, however, differs from chemicals and other manufactured products. The manufacture of software is not the production of hundreds or thousands of copies of the same end product. Software manufacturing is the ability to build repaired or enhanced versions of a given product.

The number of versions is likely to be fewer than fifty, but each repair or enhancement requires construction of a new product.

In this respect, software maintenance is similar to the production of different models of automobiles, or the inclusion of specialized features and options in a single model.¹ For such products to be successful, the effort required to build new versions, or to rebuild previous versions must be small compared to the effort invested in the research and development of the initial version.²

¹ In fact, the analogy is quite strong. Enhancements and special software products always include existing (off the shelf) components, as well as new or different components. [12]

² In cases where the initial version is a prototype this may not be exactly true -- the research and development effort might equal the production effort. But the manufacturing activity is qualitatively different from building the first prototype.

Software maintenance "is jargon for the modification of software already in service.... It is always aimed at producing something better than the original." [12, p. 214] Software maintenance is identical to other industrial engineering production processes. In all cases efficient construction of the final products is the goal of production. This is the goal of software engineering.

3. The Software Assembly Line.

Automation of software construction is a frequent theme in recent software engineering literature. [1, 2, 6, 8, 11, 12] There are no reasons why software cannot be manufactured and maintained in an assembly line fashion once an initial version is developed. [9] The process is the same as any other manufacturing process: primary components are transformed by tools and procedures into larger units that are integrated by stages into finished products. For software products the primary components are modules of source code. The tools and procedures include editors (editing), compilers (compilation), linkers and loaders (binding), and possibly delivery of running programs to target machines.

A Software Assembly Line is shown in Figure 1. Square boxes represent workbenches and integration stations. Solid arrows represent movement of finished parts between stations on the line. The actual number of programmer workbenches and integration stations depend on the size and nature of the product being manufactured and maintained. Integration stations provide places where subassemblies are built and certified. These certifications in turn provide information for monitoring progress.

Software engineering discipline is based on incremental development and testing. [2, 5, 7] Integration steps may include regression testing and other quality assurance procedures for tracking and correcting errors found during assembly of the final product. Incremental development also supports the management of software engineering activities. [4]

Increments provide criteria for milestone completion. These characteristics are inherent in a software assembly line environment.

In addition to assembly and testing, software maintenance requires storing all released versions of software products and their components. This activity requires recording changes to primary source components,

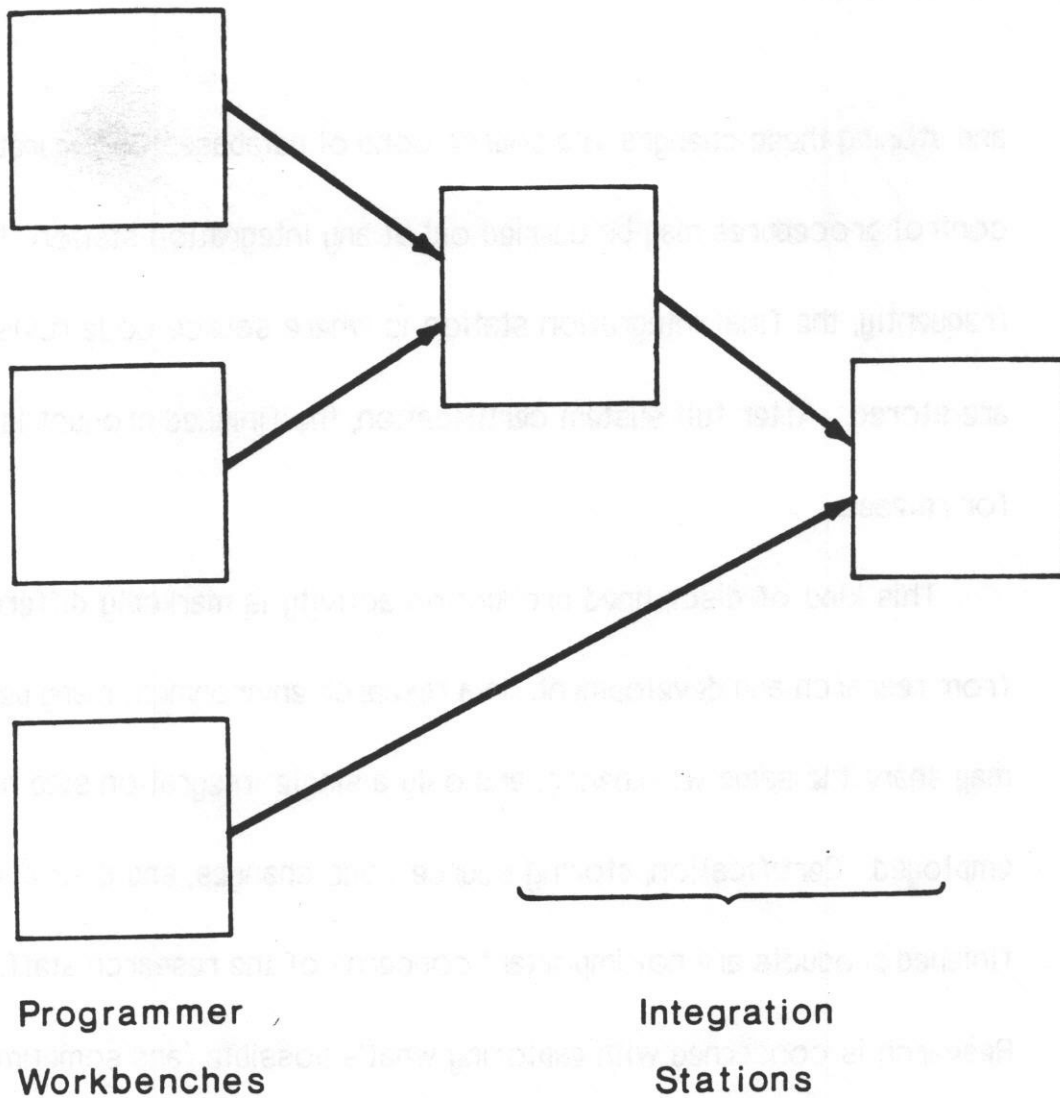


Figure 1: A Software Assembly Line.

and storing these changes in a source control database. Configuration control procedures may be carried out at any integration station. Most frequently, the final integration station is where source code revisions are stored. After full system certification, the finished product is ready for release.

This kind of disciplined production activity is markedly different from research and development. In a research environment many people may share the same workbench, and only a single integration step may be employed. Certification, storing source code changes, and distribution of finished products are not important concerns of the research staff. Research is concerned with exploring what's possible (and sometimes what's not). Once the possibilities have been discovered, the continued development and maintenance is a software engineering effort.

4. The four types of fixes.

The assembly line model provides a clear picture of how software can be manufactured and how progress can be monitored. It also provides a view of different ways in which software may be repaired. Four

different kinds of repairs are differentiated. Each repair is classified by two characteristics:

1. the place in the assembly line at which it is initiated, and
2. the number of incremental certifications to which it is subjected.

Each repair method is described in terms of three attributes:

1. the time required to complete the repair,
2. the quality and level of confidence in the repaired product,
and
3. the likelihood and scope of unexpected side effects on the
other components in the product.

Type 0: controlled fix.

A controlled fix is the textbook example of how software maintenance should be done. It is the same as a functional upgrade or enhancement. Source code modules needing modification are extracted from the source control database and delivered to individual programmer workbenches. Changes are made and individual components are tested. They are then delivered to subsequent stations on the assembly line for

integration and certification. At the final station, the repaired components are stored in the source control database, and the new version made available for distribution.

This repair method is controlled in the sense that each change is subjected to the maximum number of incremental assembly and regression testing procedures. The chances for unknown side effects appearing in the repaired product are minimized since such problems are flushed out at the preceding integration stations. Confidence in the repaired product is high.

The main drawback of a controlled fix is that the time required to complete a repair is maximized. If testing and delivery procedures are automated certification time can be reduced. But time is the price of control and frequent certification.

Type 1: direct fix.

In a direct fix, modified source components are sent directly from the programmer workbenches to a remote integration station (see Figure 2). One or more intermediate integration stations (and incremental certification steps) are skipped. In some cases repaired components are sent directly to the final integration station.

The risks inherent in this method are obvious. Changes have not had the benefit of incremental regression testing, and the likelihood of unexpected side effects is high. Since certification procedures are skipped, confidence in the reliability of the repaired system is lessened.

On the other hand the method has a high payoff. For large products, the time required to submit to all testing and integration procedures may be substantial. If the final product can be certified without delay, the time required for the repair is shortened. On the other hand, if bugs are found at the station to which the repaired components are delivered, the elided integration steps may be required to find and repair them.

There is a clear tradeoff here between quality of the repair

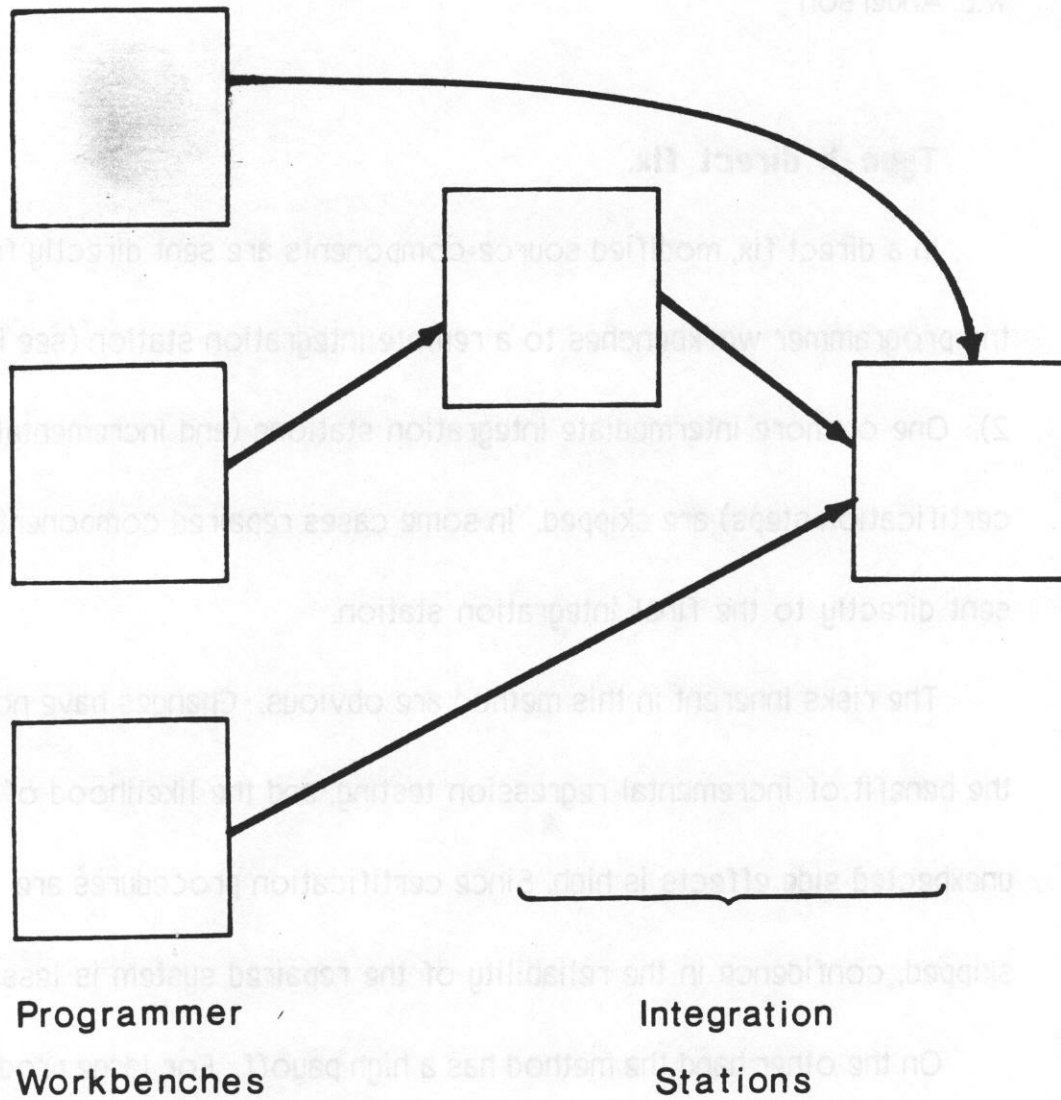


Figure 2: A direct fix.

(measured by the number of incremental certification steps) and the time available to make the repair.

Type 2: local fix.

In a local fix a programmer workbench is attached directly to an integration station (see Figure 3). Rather than delivering source code to a programmer workbench for repair, programmers make changes at a specific integration station. This always seems like a convenient procedure when the programmer workbench machines are different from those used for integration and testing. This situation is common today in view of the increased use of standalone programmer development workstations.

The same risks prevail for local and direct fixes. Although repair time is reduced, incremental certification steps are skipped and unexpected side effects are more likely. Confidence in the reliability of the repaired product is diminished.

However, local fixes have an additional problem. In controlled and direct fixes programmers work at their own workbenches. In many cases,

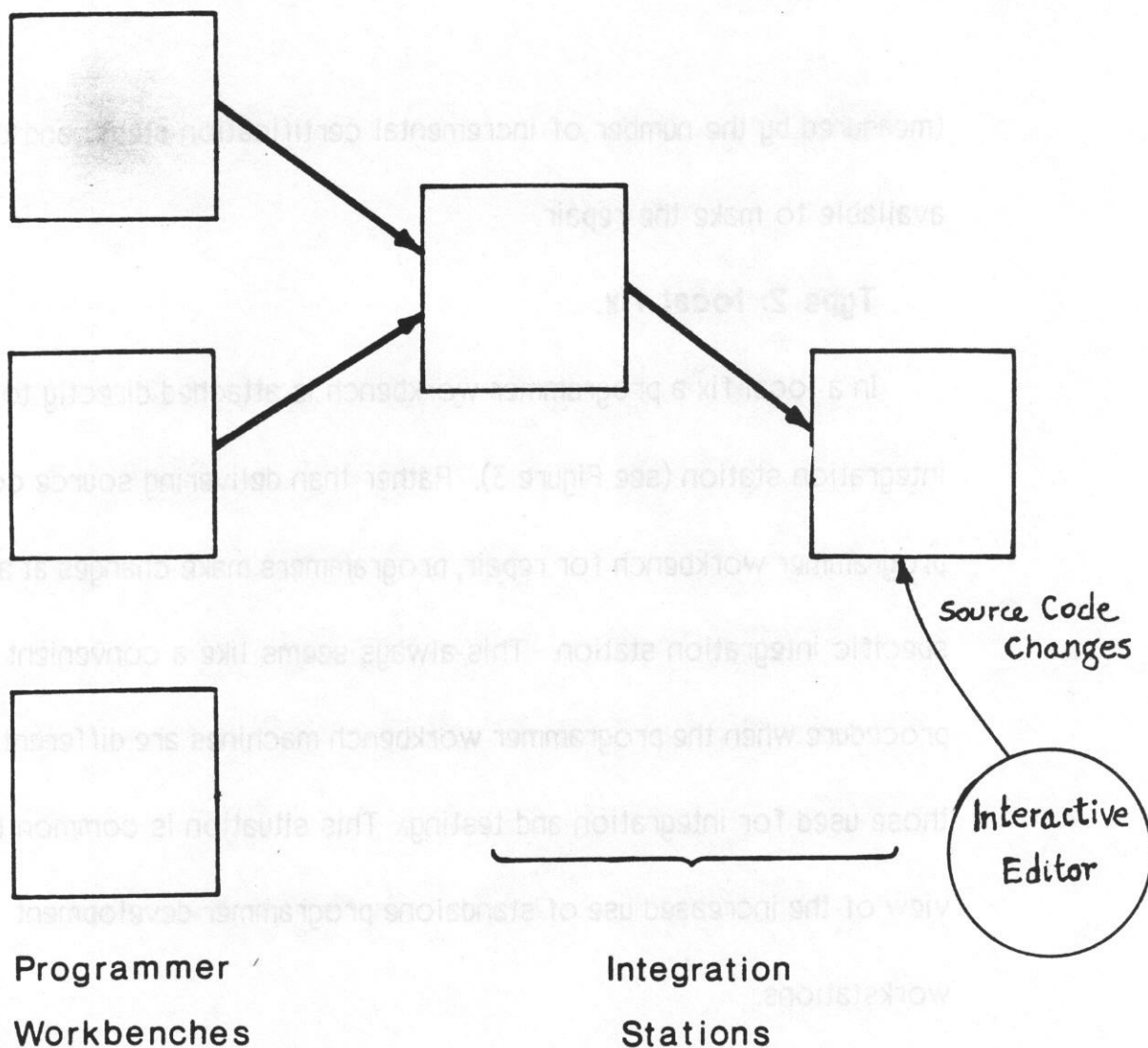


Figure 3: A local fix.

upgrades and repairs affect several components, and it is helpful for programmers to share the latest changes while testing individual components. This sharing is easily accommodated for these types of fixes, since workbenches are often on a single machine or a closely connected set of machines.

A local fix modifies components in between the customary workbench area and the controlled source code database. As a result, the changed components must be delivered to both places. Depending on the time required for these delivery procedures, local fixes might require more effort than the equivalent change handled as a direct fix.

In any event, local changes must eventually be certified in the usual way. Modified source code must submit to all incremental certification procedures. Any bugs that arise in regression testing may prove problematic to an already completed local fix. This vicious circle is well known in software maintenance activity, and it supports the statement that debugging is often rebugging. It can be particularly severe for products with a high degree of coupling among components.

Type 3: the patch.

A Type 3 fix is the ubiquitous patch of the assembled and delivered system. It usually involves direct alteration of the running system.¹ The major advantage of a patch fix is speed. The time required for repair is absolutely minimized. A patch fix is justified in emergency situations, when rapid repair is absolutely needed.

A patch fix has many drawbacks. First, since virtually all regression testing and certification is skipped, the chances of unexpected side effects are maximized. Second, the patch must be translated into source code and this requires extra effort. Third, since no source code is modified, no update of the source control database is possible. Fourth, the scope of side effects of the change cannot be known until the source code changes have been stored and distributed. The rebugging problem of the local fix is always a possible outcome. Lastly, the reliability and maintainability of the running system is called into question.

¹ In some cases a patch fix is applied to a subcomponent, such as a subroutine library, at a particular integration station. This type of local fix is considered identical to a patch fix in terms of risks and drawbacks.

In fact, a patch fix is misleading. The operating product is repaired but the maintenance activity has not begun. Maintenance is not finished until the source code has been modified and the revised product incrementally assembled and certified.

5. Conclusions for management of software maintenance.

The maintainability of a system is the probability that a failed system can be restored to operation in a specified time. It depends on product design, personnel, tools, and repair procedures. [10] The major tradeoff for software maintenance is between the time required for repair, which is often dictated by the severity of the problem, and the desire for high quality of the repaired product.

Product quality depends on how the work is done. This discussion assumes that the number of incremental integration and certification steps is directly proportional to the quality of the result. The more times the pieces are gathered into subassemblies and tested, the higher the quality of the final product. Clearly there is no guarantee that quantity insures quality. But it is reasonable to assume that a suitable

set of incremental certification procedures assures a high level of confidence in the final product.

Maintenance of software is the process of manufacturing new versions (either repairs or enhancements) of an existing software system.

This manufacturing process can be represented as an assembly line activity and this assembly line model, in turn, provides a taxonomy for differentiating among repair and enhancement methods.

The characteristics of the four repair methods are summarized in Table 1. Each method has costs and benefits. Each method provides standard procedures and criteria for completeness. These attributes permit tradeoffs based on repair urgency and certification of repair quality to be incorporated into management decisions controlling software maintenance activities.

Acknowledgments.

The four types of fixes were first proposed by M.J. McGowan III, in the course of work we did together on the Software Assembly Line. All errors and deficiencies in explanation and presentation are my own.

Table 1: The Four Types of Fixes.

<u>Repair Type</u> -----	<u>Attributes</u> -----
Controlled (0):	Maximum level of certification; Minimum unexpected side effects; Longest time to complete repair.
Direct (1), Local (2) :	Reduced level of certification; Increased likelihood of side effects; Reduced time to complete repair.
Patch (3):	Minimum level of certification; Maximum likelihood of side effects; Minimum time to complete repair.

References

1. Basset, P. and J. Giblon, "Computer Aided Programming," in Proceedings of 1983 SoftFair: A Conference on Software Development Tools, Techniques, and Alternatives, (Silver Springs, MD: IEEE Computer Society Press, 1983), pp. 9 - 20.
2. Bauer, F.L., "Software Engineering," in Software Engineering: An Advanced Course, F.L. Bauer, ed., (New York: Springer-Verlag, 1973, 1977), pp 522 - 545.
3. Boyle, James M., "Software Adaptability and Program Transformation," in Software Engineering, H. Freeman and P.M. Lewis II, eds., (New York: Academic Press, 1980), pp. 75 - 94.
4. Dyer, M., "The Management of Software Engineering, Part IV: Software Development Practices," IBM Systems Journal, **19** (4), 1980, pp. 451 - 465.
5. Giddings, Richard V., "Accommodating Uncertainty in Software Design," Comm. ACM, **27** (5), May 1984, pp. 428 - 434.
6. Haberman, A. N., "Tools for Software System Construction," in Software Development Tools, W.E. Riddle and R.E. Fairley, eds., (New York: Springer-Verlag, 1980), pp. 10 - 39.
7. Hewitt, Carl, "Evolutionary Programming," in Software Engineering, H. Freeman and P.M. Lewis II, eds., (New York: Academic Press, 1980), pp. 133 - 147.

8. Lampson, B.W. and E.E. Schmidt, "Organizing Software in a Distributed Environment," in Proceedings of SIGPLAN 83 Symposium on Programming Language Issues in Software Systems, (New York: ACM, 1983), pp. 1 - 13.
9. McGowan, M.J. III and W. L. Anderson, "Software Must Move! A Description of the Software Assembly Line," in Proceedings of 1983 SoftFair: A Conference on Software Development Tools, Techniques, and Alternatives, (Silver Springs, MD: IEEE Computer Society Press, 1983), pp. 119 - 126.
10. Ross, Harold D., Jr., "Reliability," in System Engineering Handbook, Robert E. Machol, ed., (New York: McGraw Hill Book Co., 1965), Chapter 33, p. 33-14.
11. Rowland, B. R. and R. J. Welsch, "The 3B20D Processor & DMERT Operating System: Software Development System," Bell System Technical Journal, **62** (1), Part 2, January 1983, pp. 275 - 289.
12. Tajima, D. and T. Matsubara, "Inside the Japanese Software Industry," IEEE Computer, **17** (3), March 1984, pp. 34 - 43.
13. Van Horn, Earl C., "Software Must Evolve," in Software Engineering, H. Freeman and P.M. Lewis II, eds., (New York: Academic Press, 1980), pp. 209 - 226.